

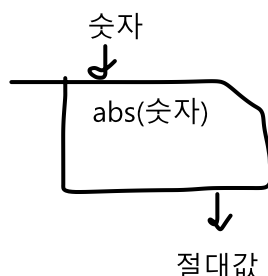
지금까지 파이썬으로 프로그래밍하기 위해 알아야 하는 대부분의 것들을 공부했다. 이제 여러분은 자신이 원하는 프로그램을 직접 만들 수 있을 것이다. 하지만 그 전에 먼저 여러분이 만들고자 하는 모듈 프로그램이 이미 만들어져 있는지 살펴보기 바란다. 물론 공부를 위해서라면 누군가 만들어 놓은 프로그램을 또 만들 수도 있다. 하지만 그런 목적이 아니라면 이미 만들어진 것을 다시 만드는 것은 불필요한 행동이다(Don't Reinvent The Wheel, 이미 있는 것을 다시 만드느라 쓸데없이 시간을 낭비하지 말라). 그리고 이미 만들어진 프로그램들은 테스트 과정을 수도 없이 거쳤기 때문에 충분히 검증되어 있다. 따라서 무엇인가 새로운 프로그램을 만들기 전에는 이미 만들어진 것들, 그중에서도 특히 **파이썬 배포본**에 함께 들어 있는 **파이썬 라이브러리**들을 살펴보는 것이 매우 중요하다.

라이브러리들을 살펴보기 전에 파이썬 내장 함수를 먼저 살펴보도록 하자. 우리는 이미 몇 가지 내장 함수들을 배웠다. print, del, type 등이 바로 그것이다. 이러한 파이썬 **내장 함수들은 외부 모듈과는 달리 import를 필요로 하지 않는다**. 아무런 설정 없이 바로 사용할 수가 있다. 이 책에서는 활용 빈도가 높고 중요한 함수들을 중심으로 알파벳 순서대로 간략히 정리했다. 파이썬으로 프로그래밍을 하기 위해 이 함수들을 지금 당장 모두 알아야 하는 것은 아니므로 가벼운 마음으로 천천히 살펴보자.

## abs

abs(x)는 어떤 숫자를 입력으로 받았을 때, 그 숫자의 절대값을 돌려주는 함수이다.

```
>>> abs(3)
3
>>> abs(-3)
3
>>> abs(-1.2)
1.2
```



## all

all(x)은 **반복 가능한(iterable) 자료형 x를 입력 인수**로 받으며, **이 x가 모두 참이면 True, 거짓이 하나라도 있으면 False**를 리턴한다.

(※ 반복 가능한 자료형이란 for문으로 그 값을 출력할 수 있는 것을 의미한다. 리스트, 튜플, 문자열, 딕셔너리, 집합 등이 있다.)

다음의 예를 보자.

```
>>> all([1, 2, 3])
True
```

0이 있는지 없는지 파악할 때 사용  
공백이 있는지 없는지 파악할 때 사용

리스트 자료형 [1, 2, 3]은 모든 요소가 참이므로 True를 리턴한다.

```
>>> all([1, 2, 3, 0])
False
```

리스트 자료형 [1, 2, 3, 0] 중에서 요소 0은 거짓이므로 False를 리턴한다.

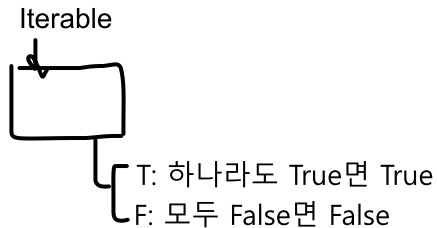
(※ 자료형의 참과 거짓에 대해 잘 기억나지 않는다면 02-7절을 다시 한 번 읽어 보자.)

## any

any(x)는 x 중 하나라도 참이 있을 경우 True를 리턴하고, x가 모두 거짓일 경우에만 False를 리턴한다. all(x)의 반대 경우라고 할 수 있다.

다음의 예를 보자.

```
all([1,2,3,0])  False
>>> any([1, 2, 3, 0])  True
True
```



리스트 자료형 [1, 2, 3, 0] 중에서 1, 2, 3이 참이므로 True를 리턴한다.

```
>>> any([0, ""])
False
```

리스트 자료형 [0, ""]의 요소 0과 ""은 모두 거짓이므로 False를 리턴한다.

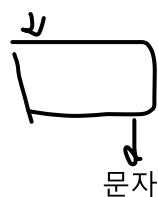
## chr

chr(i)는 아스키(ASCII) 코드값을 입력으로 받아 그 코드에 해당하는 문자를 출력하는 함수이다.

(※ 아스키 코드란 0에서 127 사이의 숫자들을 각각 하나의 문자 또는 기호에 대응시켜 놓은 것이다.)

```
>>> chr(97)
'a'
>>> chr(48)
'0'
```

아스키코드 숫자값(10진수)



기억하기  
48(0x30) -> "0"  
65(0x41) -> "A"  
97(0x61) -> "a"

## dir

dir은 객체가 자체적으로 가지고 있는 변수나 함수를 보여 준다. 아래 예는 리스트와 딕셔너리 객체의 관련 함수들(메서드)을 보여 주는 예이다. 우리가 02장에서 살펴보았던 자료형 관련 함수들을 만나볼 수 있을 것이다.

```
>>> dir([1, 2, 3])
['append', 'count', 'extend', 'index', 'insert', 'pop',...]
>>> dir({'1':'a'})
['clear', 'copy', 'get', 'has_key', 'items', 'keys',...]
```



## divmod

나눈 나머지

divmod(a, b)는 2개의 숫자를 입력으로 받는다. 그리고 a를 b로 나눈 몫과 나머지를 튜플 형태로 리턴하는 함수이다.

```
>>> divmod(7, 3)
(2, 1)
>>> divmod(1.3, 0.2)
(6.0, 0.099999999999999978)
```

## enumerate

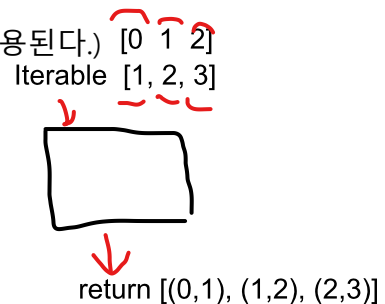
enumerate는 "열거하다"라는 뜻이다. 이 함수는 순서가 있는 자료형(리스트, 튜플, 문자열)을 입력으로 받아 인덱스 값을 포함하는 enumerate 객체를 리턴한다.

(※ 보통 enumerate 함수는 아래 예제처럼 for문과 함께 자주 사용된다.)

무슨 말인지 잘 이해되지 않으면 다음 예를 보자.

```
>>> for i, name in enumerate(['body', 'foo', 'bar']):
...     print(i, name)
...
0 body
1 foo
2 bar
```

[(0,"body"), (1,"foo"), (2,"bar")]



순서값과 함께 body, foo, bar가 순서대로 출력되었다. 즉, 위 예제와 같이 enumerate를 for문과 함께 사용하면 자료형의 현재 순서(index)와 그 값을 쉽게 알 수 있다.

for문처럼 반복되는 구간에서 객체가 현재 어느 위치에 있는지 알려주는 인덱스 값이 필요할때 enumerate 함수를 사용하면 매우 유용하다.

## eval

eval(expression)은 실행 가능한 문자열(1+2, 'hi' + 'a' 같은 것)을 입력으로 받아 문자열을 실행한 결과값을 리턴하는 함수이다.

```
>>> eval('1+2')
3
>>> eval("'hi' + 'a'")
'hia'
>>> eval('divmod(4, 3)')
(1, 1)
```

보통 eval은 입력받은 문자열로 파이썬 함수나 클래스를 동적으로 실행하고 싶은 경우에 사용된다.

## filter

첫 번째 매개변수에 함수가 들어감

filter란 무엇인가를 걸러낸다는 뜻으로, filter 함수도 동일한 의미를 가진다. filter 함수는 첫 번째 인

수로 함수 이름을, 두 번째 인수로 그 함수에 차례로 들어갈 반복 가능한 자료형을 받는다. 그리고 두 번째 인수인 반복 가능한 자료형 요소들이 첫 번째 인수인 함수에 입력되었을 때 리턴값이 참인 것만 묶어서(걸러내서) 돌려준다.

다음의 예를 보자.

```
#positive.py
def positive(l):
    result = []
    for i in l:
        if i > 0:
            result.append(i)
    return result

print(positive([1,-3,2,0,-5,6]))
```

결과값: [1, 2, 6]

즉, 위에서 만든 positive 함수는 리스트를 입력값으로 받아 각각의 요소를 판별해서 양수값만 리턴하는 함수이다.

filter 함수를 이용하면 위의 내용을 아래와 같이 간단하게 작성할 수 있다.

```
#filter1.py
def positive(x):
    return x > 0

print(list(filter(positive, [1, -3, 2, 0, -5, 6])))
```

결과값: [1, 2, 6]

여기서는 두 번째 인수인 리스트의 요소들이 첫 번째 인수인 positive 함수에 입력되었을 때 리턴값이 참인 것만 묶어서 돌려준다. 앞의 예에서는 1, 2, 6만 양수여서  $x > 0$  이라는 문장이 참이 되므로 [1, 2, 6]이라는 결과값을 리턴하게 된 것이다.

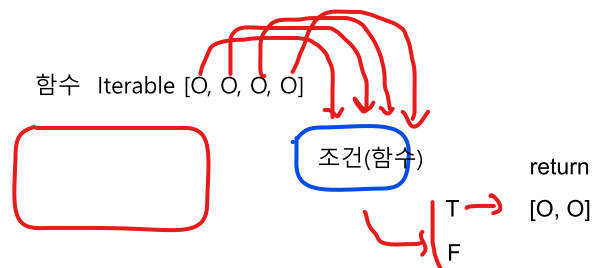
앞의 함수는 lambda를 이용하면 더욱 간편하게 코드를 작성할 수 있다. lambda 함수는 조금 뒤에서 설명한다.

```
>>> print(list(filter(lambda x: x > 0, [1, -3, 2, 0, -5, 6])))
```

## hex

hex(x)는 정수값을 입력받아 16진수(hexadecimal)로 변환하여 리턴하는 함수이다.

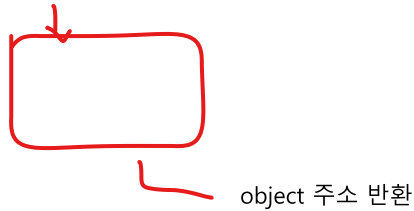
```
>>> hex(234)
'0xea'
>>> hex(3)
'0x3'
```



## id

id(object)는 객체를 입력받아 객체의 고유 주소값(레퍼런스)을 리턴하는 함수이다.

```
>>> a = 3
>>> id(3)
135072304
>>> id(a)
135072304
>>> b = a
>>> id(b)
135072304
```



위 예의 3, a, b는 고유 주소값이 모두 135072304이다. 즉, 3, a, b가 모두 같은 객체를 가리키고 있음을 알 수 있다.

만약 id(4)라고 입력하면 4는 3, a, b와 다른 객체이므로 당연히 다른 고유 주소값이 출력된다.

```
>>> id(4)
135072292
```

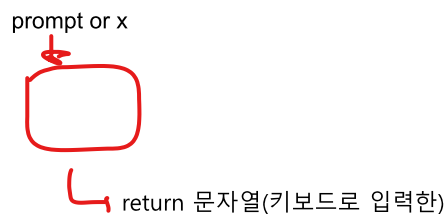
## input

input([prompt])은 사용자 입력을 받는 함수이다. 입력 인수로 문자열을 주면 아래의 세 번째 예에서 볼 수 있듯이 그 문자열은 프롬프트가 된다.

(※ [ ] 기호는 괄호 안의 내용을 생략할 수 있다는 관례적인 표기법임을 기억하자.)

(※ 파이썬 2.7 버전의 경우 input 대신 raw\_input을 사용해야 한다.)

```
>>> a = input()
hi
>>> a
'hi'
>>> b = input("Enter: ")
Enter: hi
```



위에서 입력받은 문자열을 확인해 보면 다음과 같다.

```
>>> b
'hi'
```

## int

int(x)는 문자열 형태의 숫자나 소수점이 있는 숫자 등을 정수 형태로 리턴하는 함수로, 정수를 입력으로 받으면 그대로 리턴한다.

```
>>> int('3')
3
```

```
>>> int(3.4)
3
```

int(x, radix)는 radix 진수로 표현된 문자열 x를 10진수로 변환하여 리턴한다.

2진수로 표현된 '11'의 10진수 값은 다음과 같이 구한다.

```
>>> int('11', 2)
3
```

16진수로 표현된 '1A'의 10진수 값은 다음과 같이 구한다.

```
>>> int('1A', 16)
26
```

## isinstance

isinstance(object, class)는 첫 번째 인수로 **인스턴스**, 두 번째 인수로 **클래스 이름**을 받는다. **입력으로 받은 인스턴스가 그 클래스의 인스턴스인지**를 판단하여 참이면 True, 거짓이면 False를 리턴한다.

```
>>> class Person: pass
...
>>> a = Person()
>>> isinstance(a, Person)
True
```

위의 예는 a가 Person 클래스에 의해서 생성된 인스턴스임을 확인시켜 준다.

```
>>> b = 3
>>> isinstance(b, Person)
False
```

b는 Person 클래스에 의해 생성된 인스턴스가 아니므로 False를 리턴한다.

```
def positive(x) : return x > 0    <- 함수 선언
positive()                       <- 함수 생성
```



## lambda

명령어

선언만 한 것.

간단한 코드를 주로.

lambda는 함수를 생성할 때 사용하는 **예약어**로, **def와 동일한 역할**을 한다. 보통 함수를 **한줄**로 간결하게 만들 때 사용한다. 우리말로 "람다"라고 읽고 def를 사용해야 할 정도로 복잡하지 않거나 def를 사용할 수 없는 곳에 주로 쓰인다. 사용법은 다음과 같다.

lambda 인수1, 인수2, ... : 인수를 이용한 표현식

함수를 다루는 표현식

한번 직접 만들어 보자.

def : 복잡한 함수 -> 반복 사용 가능

lambda : 간단한 함수 -> 일회성

python.org에서 쓰지 말라고 함.

```
>>> sum = lambda a, b: a+b
>>> sum(3,4)
7
```

lambda를 이용한 sum 함수는 인수로 a, b를 받아 서로 더한 값을 돌려준다. 위의 예제는 def를 사용

한 아래 함수와 하는 일이 완전히 동일하다.

```
>>> def sum(a, b):
...     return a+b
...
>>>
```

그렇다면 def가 있는데 왜 lambda라는 것이 나오게 되었을까? 이유는 간단하다. lambda는 def 보다 간결하게 사용할 수 있기 때문이다. 또한 lambda는 def를 사용할 수 없는 곳에도 사용할 수 있다. 다음 예제에서 리스트 내에 lambda가 들어간 경우를 살펴보자.

```
>>> myList = [lambda a,b:a+b, lambda a,b:a*b]
>>> myList
[at 0x811eb2c>, at 0x811eb64>]
```

즉, 리스트 각각의 요소에 lambda 함수를 만들어 바로 사용할 수 있다. 첫 번째 요소 myList[0]은 2개의 입력값을 받아 두 값의 합을 돌려주는 lambda 함수이다.

```
>>> myList[0]
at 0x811eb2c>
>>> myList[0](3,4)
7
```

두 번째 요소 myList[1]은 2개의 입력값을 받아 두 값의 곱을 돌려주는 lambda 함수이다.

```
>>> myList[1](3,4)
12
```

파이썬에 익숙해질수록 lambda 함수가 굉장히 편리하다는 사실을 알게 될 것이다.

## len

len(s)은 입력값 s의 길이(요소의 전체 개수)를 리턴하는 함수이다.

```
>>> len("python")
6
>>> len([1,2,3])
3
>>> len((1, 'a'))
2
len(Iterable)
```

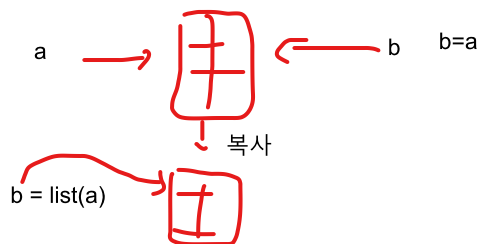
## list

list(s)는 반복 가능한 자료형 s를 입력받아 리스트로 만들어 리턴하는 함수이다.

```
>>> list("python")
['p', 'y', 't', 'h', 'o', 'n']
>>> list((1,2,3)) 수정 불가
[1, 2, 3] 수정 가능
```

list 함수에 리스트를 입력으로 주면 똑같은 리스트를 복사하여 돌려준다.

```
>>> a = [1, 2, 3]
>>> b = list(a)
>>> b
[1, 2, 3]
```



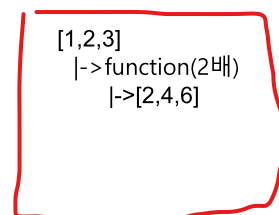
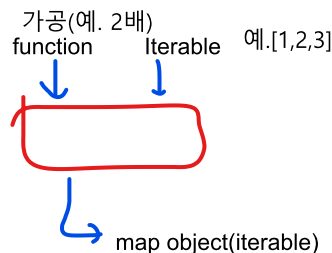
## map

map(f, iterable)은 함수(f)와 반복 가능한(iterable) 자료형을 입력으로 받는다. map은 입력받은 자료형의 각 요소가 함수 f에 의해 수행된 결과를 묶어서 리턴하는 함수이다.

다음의 예를 보자.

```
# two_times.py
def two_times(numberList):
    result = [ ]
    for number in numberList:
        result.append(number*2)
    return result
```

```
result = two_times([1, 2, 3, 4])
print(result)
```



two\_times 함수는 리스트 요소를 입력받아 각 요소에 2를 곱한 결과값을 돌려준다. 실행 결과는 다음과 같다.

결과값: [2, 4, 6, 8]

위의 예제는 map 함수를 이용하면 다음처럼 바꿀 수 있다.

```
>>> def two_times(x): return x*2
...
>>> list(map(two_times, [1, 2, 3, 4]))
[2, 4, 6, 8]
```

이제 앞 예제를 해석해 보자. 먼저 리스트의 첫 번째 요소인 1이 two\_times 함수의 입력값으로 들어가고, 1 \* 2의 과정을 거쳐서 2가 된다. 다음으로 리스트의 두 번째 요소인 2가 2 \* 2의 과정을 거쳐 4가 된다. 따라서 결과값 리스트는 이제 [2, 4]가 된다. 총 4개의 요소값이 모두 수행되면 최종적으로 [2, 4, 6, 8]이 리턴된다. 이것이 map 함수가 하는 일이다.

(※ 위 예에서 map의 결과를 리스트로 보여 주기 위해 list 함수를 이용하여 출력하였다. 파이썬 2.7은 map의 결과가 리스트이므로 위 예에서 list 함수를 이용하여 리스트로 변환하지 않아도 된다.)

앞의 예는 lambda를 사용하면 다음처럼 간략하게 만들 수 있다.

```
>>> list(map(lambda a: a*2, [1, 2, 3, 4]))
[2, 4, 6, 8]
```

map 함수 예를 하나 더 살펴보자.



```
# map_test.py
def plus_one(x):
    return x+1
print(list(map(plus_one, [1, 2, 3, 4, 5])))
결과값: [2, 3, 4, 5, 6]
```

위 예는 map과 plus\_one 함수를 이용하여 리스트의 각 요소값을 1씩 증가시키는 예제이다.

## max

max(iterable)는 인수로 반복 가능한 자료형을 입력받아 그 최대값을 리턴하는 함수이다.

```
>>> max([1, 2, 3])
3
>>> max("python")
'y'
```

## min

min(iterable)은 max 함수와 반대로, 인수로 반복 가능한 자료형을 입력받아 그 최소값을 리턴하는 함수이다.

```
>>> min([1, 2, 3])
1
>>> min("python")
'h'
```

## oct

oct(x)는 정수 형태의 숫자를 8진수 문자열로 바꾸어 리턴하는 함수이다.

```
>>> oct(34)
'0o42'
>>> oct(12345)
'0o30071'
```

## open

open(filename, [mode])은 "파일 이름"과 "읽기 방법"을 입력받아 **파일 객체를 리턴하는 함수**이다. 읽기 방법(mode)이 생략되면 기본값인 읽기 전용 모드(r)로 파일 객체를 만들어 리턴한다.

| mode | 설명           |
|------|--------------|
| w    | 쓰기 모드로 파일 열기 |
| r    | 읽기 모드로 파일 열기 |

wb,rb,ab

|   |                |
|---|----------------|
| a | 추가 모드로 파일 열기   |
| b | 바이너리 모드로 파일 열기 |

이미지, 음성, 데이터 파일 등(텍스트 제외)

b는 w, r, a와 함께 사용된다.

```
>>> f = open("binary_file", "rb")
```

바이너리 파일 읽기

위 예의 rb는 "바이너리 읽기 모드"를 의미한다.

아래 예의 fread와 fread2는 동일한 방법이다.

```
>>> fread = open("read_mode.txt", 'r')
>>> fread2 = open("read_mode.txt")
```

즉, 모드 부분이 생략되면 기본값으로 읽기 모드인 r을 갖게 된다.

다음은 추가 모드(a)로 파일을 여는 예이다.

```
>>> fappend = open("append_mode.txt", 'a')
```

## ord

ord(c)는 문자의 아스키 코드값을 리턴하는 함수이다.

(※ ord 함수는 chr 함수와 반대이다.)

```
>>> ord('a')
97
>>> ord('0')
48
```

## pow

pow(x, y)는 x의 y 제곱한 결과값을 리턴하는 함수이다.

```
>>> pow(2, 4)    2^4
16
>>> pow(3, 3)    3^3
27
```

## range

range([start,] stop [,step])는 for문과 함께 자주 사용되는 함수이다. 이 함수는 입력받은 숫자에 해당하는 범위의 값을 반복 가능한 객체로 만들어 리턴한다.

## 인수가 하나일 경우

시작 숫자를 지정해 주지 않으면 range 함수는 0부터 시작한다.

```
>>> list(range(5))  
[0, 1, 2, 3, 4]
```

## 인수가 2개일 경우

입력으로 주어지는 2개의 인수는 시작 숫자와 끝 숫자를 나타낸다. 단, 끝 숫자는 해당 범위에 포함되지 않는다는 것에 주의하자.

```
>>> list(range(5, 10))  
[5, 6, 7, 8, 9]
```

## 인수가 3개일 경우

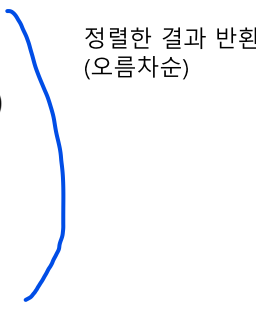
세 번째 인수는 숫자 사이의 거리를 말한다.

```
>>> list(range(1, 10, 2))  
[1, 3, 5, 7, 9]  
>>> list(range(0, -10, -1))  
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
```

## sorted

sorted(iterable) 함수는 입력값을 정렬한 후 그 결과를 리스트로 리턴하는 함수이다.

```
>>> sorted([3, 1, 2])  
[1, 2, 3]  
>>> sorted(['a', 'c', 'b'])  
['a', 'b', 'c']  
>>> sorted("zero")  
['e', 'o', 'r', 'z']  
>>> sorted((3, 2, 1))  
[1, 2, 3]
```



정렬한 결과 반환  
(오름차순)

메소드

리스트 자료형에도 sort라는 함수가 있다. 하지만 리스트 자료형의 sort 함수는 리스트 객체 그 자체를 정렬만 할 뿐 정렬된 결과를 리턴하지는 않는다.

다음 예제로 sorted 함수와 리스트 자료형의 sort 함수의 차이점을 확인해 보자.

```
>>> a = [3, 1, 2]  
>>> result = a.sort() 메서드  
>>> print(result)  
None 메서드는 정렬 결과 반환 x  
>>> a  
[1, 2, 3]
```

sort 함수는 리턴값이 없기 때문에 result 변수에 저장되는 값이 없다. 따라서 print(result)를 하면 None이 출력된다. sort 함수를 수행한 후 리턴값은 없지만 리스트 객체 a를 확인하면 [3, 1, 2]가 [1,

2, 3]으로 정렬된 것을 볼 수 있다.

## str

str(object)은 문자열 형태로 객체를 변환하여 리턴하는 함수이다.

```
>>> str(3)
'3'
>>> str('hi')
'hi'
>>> str('hi').upper()
'HI'
```

## tuple

매개변수(자료)

tuple(iterable)은 반복 가능한 자료형을 입력받아 튜플 형태로 바꾸어 리턴하는 함수이다. 만약 튜플이 입력으로 들어오면 그대로 리턴한다.

```
>>> tuple("abc")
('a', 'b', 'c')
>>> tuple([1, 2, 3])
(1, 2, 3)
>>> tuple((1, 2, 3))
(1, 2, 3)
```

iterable    반복가능

상속 ↑

list

반복가능, 모든 타입 여러 개 저장

상속 ↑

tuple

반복가능, 모든 타입 여러 개 저장, 불변

상속 ↑

string

반복가능, 모든 타입 여러 개 저장, 불변, 문자만 저장

## type

type(object)은 입력값의 자료형이 무엇인지 알려주는 함수이다.

```
>>> type("abc")
<class 'str'>
>>> type([ ])
<class 'list'>
>>> type(open("test", 'w'))
<class '_io.TextIOWrapper'>
```

object -> class 확인

[ ] -> 기호 -> 명령 -> list object 생성하라는



## zip

\*iterable 가변 매개변수  
zip(iterable\*)은 동일한 개수로 이루어진 자료형을 묶어 주는 역할을 하는 함수이다.

튜플로 반환

잘 이해되지 않는다면 다음 예제를 살펴보자.

예. 리스트 학생 점수

```
>>> list(zip([1, 2, 3], [4, 5, 6]))
[(1, 4), (2, 5), (3, 6)]
>>> list(zip([1, 2, 3], [4, 5, 6], [7, 8, 9]))
[(1, 4, 7), (2, 5, 8), (3, 6, 9)]
>>> list(zip("abc", "def"))
[('a', 'd'), ('b', 'e'), ('c', 'f')]
```

```
names = ["홍", "고", "마"]
kors = [100, 90, 80]
```

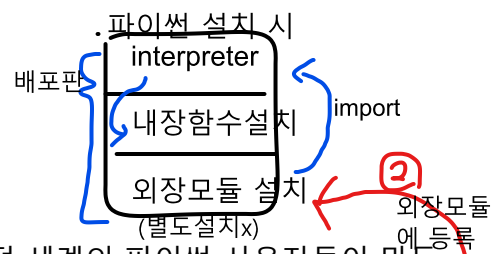
```
list(zip(names, kors))
```

```
[("홍", 100), ("고", 90), ("마", 80)]
```

## 05-6 파이썬 [외장 함수]

모듈

function  
class  
variable



이제 파이썬 프로그래밍 능력을 높여 줄 더 큰 날개를 달아 보자. 전 세계의 파이썬 사용자들이 만든 유용한 프로그램들을 모아 놓은 것이 바로 파이썬 라이브러리이다. "라이브러리"는 "도서관"이라는 뜻 그대로 원하는 정보를 찾아보는 곳이다. 모든 라이브러리를 다 알 필요는 없고 어떤 일을 할 때 어떤 라이브러리를 사용해야 한다는 정도만 알면 된다. 그러기 위해 어떤 라이브러리들이 존재하고 어떻게 사용되는지 알아야 할 필요가 있다. 자주 사용되고 꼭 알아두면 좋은 라이브러리들을 중심으로 하나씩 살펴보자. 필요한 라이브러리를 찾을 수 있어야 한다.

(※ 파이썬 라이브러리는 파이썬 설치 시 자동으로 컴퓨터에 설치가 된다.)



### sys

sys 모듈은 파이썬 인터프리터가 제공하는 변수들과 함수들을 직접 제어할 수 있게 해주는 모듈이다. (실행기)

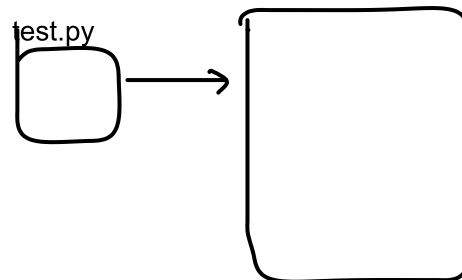
명령 행에서 인수 전달하기 - sys.argv ( ) 없으니 인자값들(리스트)을 저장하고 있는 변수

C:/User/home>python test.py abc pey guido 실행에 필요한 재료들  
파이썬 인터프리터 실행된 후 sys.argv 가능

도스 창에서 위의 예처럼 test.py 뒤에 또 다른 값들을 함께 넣어 주면 sys.argv라는 리스트에 그 값들이 추가된다.

예제를 따라 하며 확인해 보자. 우선 다음과 같은 파이썬 프로그램을 작성하자. argv\_test.py 파일은 C:/Python/Mymodules 디렉터리에 저장했다고 가정한다(만약 C:/Python/Mymodules 디렉터리가 없다면 먼저 생성하고 진행하자).

```
# argv_test.py
import sys
print(sys.argv)
```



도스 창에서 다음과 같이 실행해 보자.

C:/Python/Mymodules>python argv\_test.py you need python

['argv\_test.py', 'you', 'need', 'python']

you need python은 argv\_test.py 실행에 필요한 값이 됨.

python이라는 명령어 뒤의 모든 것들이 공백을 기준으로 나뉘어서 sys.argv 리스트의 요소가 됨을 알 수 있다.

(※ 도스 창에서는 /, W든 상관없지만, 소스코드 안에서는 반드시 / 또는 WW 기호를 사용해야 한다.)

### 강제로 스크립트 종료하기 - sys.exit

```
>>> sys.exit()
```

소유

실행 종료

sys.exit는 Ctrl+Z나 Ctrl+D를 눌러서 대화형 인터프리터를 종료하는 것과 같은 기능을 한다. 프로그램

램 파일 내에서 사용하면 프로그램을 중단시킨다.

## 자신이 만든 모듈 불러와 사용하기 - sys.path [ ] <- 리스트니 append 가능

설치된 외장 모듈

sys.path는 파이썬 모듈들이 저장되어 있는 위치를 나타낸다. 즉, 이 위치에 있는 파이썬 모듈들은 경로에 상관없이 어디에서나 불러올 수가 있다.

다음은 그 실행 결과이다.

```
>>> import sys
>>> sys.path
['', 'C:\\Windows\\SYSTEM32\\python35.zip', 'c:\\Python35\\DLLs',
'c:\\Python35\\lib', 'c:\\Python35', 'c:\\Python35\\lib\\site-packages']
>>>
```

별도로 install 한 게 저장되는 위치

위의 예에서 "는 현재 디렉터리를 말한다.

```
# path_append.py
import sys
sys.path.append("C:/Python/Mymodules")
```

위와 같이 파이썬 프로그램 파일에서 sys.path.append를 이용해 경로명을 추가할 수 있다. 이렇게 하고 난 후에는 C:/Python/Mymodules라는 디렉터리에 있는 파이썬 모듈을 불러와서 사용할 수가 있다.

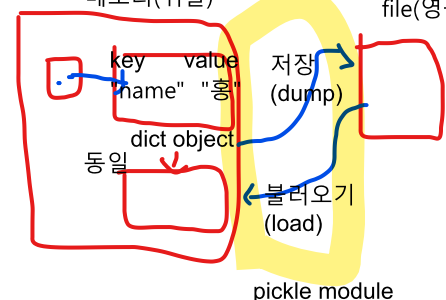


## pickle

pickle은 객체의 형태를 그대로 유지하면서 파일에 저장하고 불러올 수 있게 하는 모듈이다. 다음 예는 pickle 모듈의 dump 함수를 이용하여 딕셔너리 객체인 data를 그대로 파일에 저장하는 방법을 보여 준다.

```
>>> import pickle
>>> f = open("test.txt", 'wb')
>>> data = {1: 'python', 2: 'you need'}
>>> pickle.dump(data, f)  object를 file에 저장 (모든 object 가능)
>>> f.close()
```

data\_dict = {'name': '홍'} →



다음은 pickle.dump에 의해 저장된 파일을 pickle.load를 이용해서 원래 있던 딕셔너리 객체(data) 상태 그대로 불러오는 예이다.

```
>>> import pickle
>>> f = open("test.txt", 'rb')
>>> data = pickle.load(f)  file을 타입 그대로 불러오기
>>> print(data)
{2:'you need', 1:'python'}
```

위의 예에서는 딕셔너리 객체를 이용하였지만 어떤 자료형이든 상관없이 저장하고 불러올 수 있다.

OS 모듈은 **환경 변수**나 디렉터리, 파일 등의 OS 자원을 제어할 수 있게 해주는 모듈이다.  
운영체제가 사용하는 변수(저장)

### 내 시스템의 환경 변수값을 알고 싶을 때 - `os.environ`

시스템은 제각기 다른 환경 변수값을 가지고 있는데, `os.environ`은 현재 시스템의 환경 변수 값들을 보여 준다. 다음을 따라 해보자.

```
>>> import os
>>> os.environ
environ({'PROGRAMFILES': 'C:\\WWProgram Files', 'APPDATA': ... 생략 ...})
>>>
```

위의 결과값은 필자의 시스템 정보이다. `os.environ`은 환경 변수에 대한 정보를 딕셔너리 객체로 리턴한다. 자세히 보면 여러 가지 유용한 정보를 찾을 수 있다.

리턴받은 객체가 딕셔너리이기 때문에 다음과 같이 호출할 수 있다. 다음은 필자 시스템의 PATH 환경 변수에 대한 내용이다.

```
>>> os.environ['PATH']
'C:\\WWProgramData\\WWOracle\\WWJava\\WWjavapath;...생략...'
```

### 디렉터리 위치 변경하기 - `os.chdir`

`os.chdir`을 이용하면 아래와 같이 현재 디렉터리의 위치를 변경할 수 있다.

```
>>> os.chdir("C:\\WINDOWS")
```

### 디렉터리 위치 리턴받기 - `os.getcwd`

`os.getcwd`는 현재 자신의 디렉터리 위치를 리턴한다.

```
>>> os.getcwd()
'C:\\WINDOWS'
```

### 시스템 명령어 호출하기 - `os.system`

시스템 자체의 프로그램이나 기타 명령어들을 파이썬에서 호출할 수도 있다. `os.system("명령어")`처럼 사용한다. 다음은 현재 디렉터리에서 시스템 명령어인 `dir`을 실행하는 예이다.

```
>>> os.system("dir")
```

### 실행한 시스템 명령어의 결과값 리턴받기 - `os.popen`

`os.popen`은 시스템 명령어를 실행시킨 결과값을 읽기 모드 형태의 파일 객체로 리턴한다.

```
>>> f = os.popen("dir")
```

읽어 들인 파일 객체의 내용을 보기 위해서는 다음과 같이 하면 된다.

```
>>> print(f.read())
```

## 기타 유용한 os 관련 함수

| 함수                  | 설명                                   |
|---------------------|--------------------------------------|
| os.mkdir(디렉터리)      | 디렉터리를 생성한다.                          |
| os.rmdir(디렉터리)      | 디렉터리를 삭제한다. 단, 디렉터리가 비어있어야 삭제가 가능하다. |
| os.unlink(파일)       | 파일을 지운다.                             |
| os.rename(src, dst) | src라는 이름의 파일을 dst라는 이름으로 바꾼다.        |

## shutil

shutil은 파일을 복사해 주는 파이썬 모듈이다.

### 파일 복사하기 - shutil.copy(src, dst)

src라는 이름의 파일을 dst로 복사한다. 만약 dst가 디렉터리 이름이라면 src라는 파일 이름으로 dst라는 디렉터리에 복사하고 동일한 파일 이름이 있을 경우에는 덮어쓴다.

다음의 예를 보자.

```
>>> import shutil
>>> shutil.copy("src.txt", "dst.txt")
```

위 예를 실행해 보면 src.txt 파일과 동일한 내용의 파일이 dst.txt로 복사되는 것을 확인할 수 있다.

## glob

가끔 파일을 읽고 쓰는 기능이 있는 프로그램을 만들다 보면 특정 디렉터리에 있는 파일 이름을 모두 알아야 할 때가 있다. 그럴 때 사용하는 모듈이 바로 glob이다.

### 디렉터리에 있는 파일들을 리스트로 만들기 - glob(pathname)

glob 모듈은 디렉터리 내의 파일들을 읽어서 리턴한다. \*, ? 등의 메타 문자를 써서 원하는 파일만 읽어 들일 수도 있다.

다음은 C:/Python이라는 디렉터리에 있는 파일 중 이름이 문자 q로 시작하는 파일들을 모두 찾아서 읽어 들이는 예이다.



```
>>> import glob
>>> glob.glob("C:/Python/q*")
['C:\\Python\\Quiz.py', 'C:\\Python\\Quiz.py.bak']
>>>
```

## tempfile

파일을 임시로 만들어서 사용할 때 유용한 모듈이 바로 tempfile이다. tempfile.mktemp()는 중복되지 않는 임시 파일의 이름을 무작위로 만들어서 리턴한다.

```
>>> import tempfile
>>> filename = tempfile.mktemp()
>>> filename
'C:\\WINDOWS\\TEMP\\~275151-0'
```

tempfile.TemporaryFile()은 임시 저장 공간으로 사용될 파일 객체를 리턴한다. 이 파일은 기본적으로 바이너리 쓰기 모드(wb)를 갖는다. f.close()가 호출되면 이 파일 객체는 자동으로 사라진다.

```
>>> import tempfile
>>> f = tempfile.TemporaryFile()
>>> f.close()
```

## time

시간과 관련된 time 모듈에는 유용한 함수가 굉장히 많다. 그중에서 가장 유용한 몇 가지만 알아보자.

### time.time

time.time()은 UTC(Universal Time Coordinated 협정 세계 표준시)를 이용하여 현재 시간을 실수 형태로 리턴하는 함수이다. 1970년 1월 1일 0시 0분 0초를 기준으로 지난 시간을 초 단위로 리턴한다.

```
>>> import time
>>> time.time()
988458015.73417199
```

### time.localtime

time.localtime은 time.time()에 의해서 반환된 실수값을 이용해서 연도, 월, 일, 시, 분, 초,.. 의 형태로 바꾸어 주는 함수이다.

```
>>> time.localtime(time.time())
time.struct_time(tm_year=2013, tm_mon=5, tm_mday=21, tm_hour=16,
                  tm_min=48, tm_sec=42, tm_wday=1, tm_yday=141, tm_isdst=0)
```

### time.asctime

위의 time.localtime에 의해서 반환된 튜플 형태의 값을 인수로 받아서 날짜와 시간을 알아보기 쉬운 형태로 리턴하는 함수이다.

```
>>> time.asctime(time.localtime(time.time()))
'Sat Apr 28 20:50:20 2001'
```

## time.ctime

time.asctime(time.localtime(time.time()))은 time.ctime()을 이용해 간편하게 표시할 수 있다. asctime과 다른점은 ctime은 항상 현재 시간만을 리턴한다는 점이다.

```
>>> time.ctime()
'Sat Apr 28 20:56:31 2001'
```

## time.strftime

time.strftime('출력할 형식 포맷 코드', time.localtime(time.time()))

strftime 함수는 시간에 관계된 것을 세밀하게 표현할 수 있는 여러 가지 포맷 코드를 제공한다.

*시간에 관계된 것을 표현하는 포맷 코드*

| 포맷코드 | 설명                  | 예                 |
|------|---------------------|-------------------|
| %a   | 요일 줄임말              | Mon               |
| %A   | 요일                  | Monday            |
| %b   | 달 줄임말               | Jan               |
| %B   | 달                   | January           |
| %c   | 날짜와 시간을 출력함         | 06/01/01 17:22:21 |
| %d   | 날(day)              | [00,31]           |
| %H   | 시간(hour)-24시간 출력 형태 | [00,23]           |
| %I   | 시간(hour)-12시간 출력 형태 | [01,12]           |
| %j   | 1년 중 누적 날짜          | [001,366]         |
| %m   | 달                   | [01,12]           |
| %M   | 분                   | [01,59]           |
| %p   | AM or PM            | AM                |
| %S   | 초                   | [00,61]           |
| %U   | 1년 중 누적 주-일요일을 시작으로 | [00,53]           |
| %w   | 숫자로 된 요일            | [0(일요일),6]        |
| %W   | 1년 중 누적 주-월요일을 시작으로 | [00,53]           |

|    |                       |          |
|----|-----------------------|----------|
| %x | 현재 설정된 로케일에 기반한 날짜 출력 | 06/01/01 |
| %X | 현재 설정된 로케일에 기반한 시간 출력 | 17:22:21 |
| %Y | 년도 출력                 | 2001     |
| %Z | 시간대 출력                | 대한민국 표준시 |
| %% | 문자                    | %        |
| %y | 세기부분을 제외한 년도 출력       | 01       |

다음은 time.strptime을 사용하는 예이다.

```
>>> import time
>>> time.strptime('%x', time.localtime(time.time()))
'05/01/01'
>>> time.strptime('%c', time.localtime(time.time()))
'05/01/01 17:22:21'
```

## time.sleep

time.sleep 함수는 주로 루프 안에서 많이 사용된다. 이 함수를 사용하면 일정한 시간 간격을 두고 루프를 실행할 수 있다. 다음의 예를 보자.

```
#sleep1.py
import time
for i in range(10):
    print(i)
    time.sleep(1)
```

위 예는 1초 간격으로 0부터 9까지의 숫자를 출력한다. 위 예에서 볼 수 있듯이 time.sleep 함수의 인수는 실수 형태를 쓸 수 있다. 즉, 1이면 1초, 0.5면 0.5초가 되는 것이다.

## calendar

calendar는 파이썬에서 달력을 볼 수 있게 해주는 모듈이다.

**calendar.calendar(연도)**로 사용하면 그 해의 전체 달력을 볼 수 있다. 결과값은 달력이 너무길어 생략하겠다.

```
>>> import calendar
>>> print(calendar.calendar(2015))
```

calendar.prcal(연도)를 사용해도 위와 똑같은 결과값을 얻을 수 있다.

```
>>> calendar.prcal(2015)
```

다음의 예는 2015년 12월의 달력만 보여 준다.

```
>>> calendar.prmonth(2015, 12)
```

December 2015  
Mo Tu We Th Fr Sa Su  
2 3 4 5 6  
7 8 9 10 11 12 13  
14 15 16 17 18 19 20  
21 22 23 24 25 26 27  
28 29 30 31

### calendar.weekday

calendar 모듈의 또 다른 유용한 함수를 보자. weekday(연도, 월, 일) 함수는 그 날짜에 해당하는 요일 정보를 리턴한다. 월요일은 0, 화요일은 1, 수요일은 2, 목요일은 3, 금요일은 4, 토요일은 5, 일요일은 6이라는 값을 리턴한다.

```
>>> calendar.weekday(2015, 12, 31)
3
```

위의 예에서 2015년 12월 31일은 목요일임을 보여 준다.

### calendar.monthrange

monthrange(연도, 월) 함수는 입력받은 달의 1일이 무슨 요일인지와 그 달이 며칠까지 있는지를 튜플 형태로 리턴한다.

```
>>> calendar.monthrange(2015, 12)
(1, 31)
```

위의 예는 2015년 12월의 1일은 화요일이고, 이 달은 31일까지 있다는 것을 보여 준다.

날짜와 관련된 프로그래밍을 할 때 위의 2가지 함수는 매우 유용하게 사용된다.

## random

random은 난수(규칙이 없는 임의의 수)를 발생시키는 모듈이다. random과 randint에 대해서 알아보자.

다음은 0.0에서 1.0 사이의 실수 중에서 난수값을 리턴하는 예를 보여 준다.

```
>>> import random
>>> random.random()
0.53840103305098674
```

다음 예는 1에서 10 사이의 정수 중에서 난수값을 리턴한다.

```
>>> random.randint(1, 10)
6
```

다음 예는 1에서 55 사이의 정수 중에서 난수값을 리턴한다.

```
>>> random.randint(1, 55)
43
```

random 모듈을 이용해서 재미있는 함수를 하나 만들어 보자.

```
# random_pop.py
import random
def random_pop(data):
    number = random.randint(0, len(data)-1)
    return data.pop(number)

if __name__ == "__main__":
    data = [1, 2, 3, 4, 5]
    while data: print(random_pop(data))
```

결과값:

```
2
3
1
5
4
```

위의 random\_pop 함수는 리스트의 요소 중에서 무작위로 하나를 선택하여 꺼낸 다음 그 값을 리턴한다. 물론 꺼내진 요소는 pop 메서드에 의해 사라진다.

random\_pop 함수는 random 모듈의 choice 함수를 사용하여 다음과 같이 좀 더 직관적으로 만들 수도 있다.

```
def random_pop(data):
    number = random.choice(data)
    data.remove(number)
    return number
```

random.choice 함수는 입력으로 받은 리스트에서 무작위로 하나를 선택하여 리턴한다.

리스트의 항목을 무작위로 섞고 싶을 때는 random.shuffle 함수를 이용하면 된다.

```
>>> import random
>>> data = [1, 2, 3, 4, 5]
>>> random.shuffle(data)
>>> data
[5, 1, 3, 4, 2]
>>>
```

[1, 2, 3, 4, 5]라는 리스트가 shuffle 함수에 의해 섞여서 [5, 1, 3, 4, 2]로 변한 것을 확인할 수 있다.

## webbrowser

webbrowser는 자신의 시스템에서 사용하는 기본 웹 브라우저가 자동으로 실행되게 하는 모듈이다. 아래의 예제는 웹 브라우저를 자동으로 실행시키고 해당 URL인 <http://google.com>으로 가게 해준다.

```
>>> import webbrowser
>>> webbrowser.open("http://google.com")
```

webbrowser의 open 함수는 웹 브라우저가 이미 실행된 상태이면 입력 주소로 이동한다. 만약 웹 브라우저가 실행되지 않은 상태이면 새로 웹 브라우저를 실행한 후 해당 주소로 이동한다.

open\_new 함수는 이미 웹 브라우저가 실행된 상태이더라도 새로운 창으로 해당 주소가 열리도록 한다.

```
>>> webbrowser.open_new("http://google.com")
```

## [스레드를 다루는 threading 모듈]

컴퓨터에서 동작하고 있는 프로그램을 프로세스(Process)라고 한다. 보통 1개의 프로세스는 1가지 일만 하지만, 스레드를 이용하면 한 프로세스 내에서 2가지 또는 그 이상의 일을 동시에 수행하게 할 수 있다. 간단한 예제로 설명을 대신하겠다.

```
import threading
import time

def say(msg):
    while True:
        time.sleep(1)
        print(msg)

for msg in ['you', 'need', 'python']:
    t = threading.Thread(target=say, args=(msg,))
    t.daemon = True
    t.start()

for i in range(100):
    time.sleep(0.1)
    print(i)
```

첫 번째 for문에서 ['you', 'need', 'python']이라는 리스트의 요소 개수만큼 스레드가 생성되고, 생성된 스레드는 say 메서드를 수행하게 되어 1초에 한 번씩 입력으로 받은 msg 변수값을 리턴한다. 두 번째 for문은 매 0.1초마다 0부터 99까지 숫자를 출력하는데, 바로 이 부분이 메인 프로그램이 되며 이 메인 프로그램이 종료되는 순간 생성된 스레드들도 함께 종료가 된다. t.daemon = True와 같이 daemon 플래그를 설정하면 주 프로그램이 종료되는 순간 데몬 스레드도 함께 종료된다.

위 예제의 실행 결과값은 다음과 비슷할 것이다.

```
0
you
need
python
1
2
3
4
```

```
5
6
7
8
9
10
you
need
python
11
12
...
```

위 결과값에서 볼 수 있듯이 스레드는 메인 프로그램과는 별도로 실행되는 것을 확인할 수 있다.

이러한 스레드 프로그래밍을 가능하게 해주는 것이 바로 `threading.Thread` 클래스이다. 이 클래스의 첫번째 인수는 함수 이름을, 두 번째 인수는 첫 번째 인수인 함수의 입력 변수를 받는다. 다음과 같이 스레드를 클래스로 정의해도 동일한 결과를 얻을 수 있다.

```
import threading
import time

class MyThread(threading.Thread):
    def __init__(self, msg):
        threading.Thread.__init__(self)
        self.msg = msg
        self.daemon = True

    def run(self):
        while True:
            time.sleep(1)
            print(self.msg)

for msg in ['you', 'need', 'python']:
    t = MyThread(msg)
    t.start()

for i in range(100):
    time.sleep(0.1)
    print(i)
```

스레드를 클래스로 정의할 경우에는 `__init__` 메서드에서 `threading.Thread.__init__(self)`와 같이 부모 클래스의 생성자를 반드시 호출해야 한다. `MyThread`로 생성된 객체의 `start` 메서드를 실행할 때는 `MyThread` 클래스의 `run` 메서드가 자동으로 수행된다.

## 06-1 정규 표현식 살펴보기

정규 표현식(Regular Expressions)은 복잡한 문자열을 처리할 때 사용하는 기법으로, 파이썬만의 고유 문법이 아니라 문자열을 처리하는 모든 곳에서 사용된다. 정규 표현식을 배우는 것은 파이썬을 배우는 것과는 또 다른 영역의 과제이다.

(※ 정규 표현식은 줄여서 간단히 "정규식"이라고도 말한다.)

### 정규 표현식은 왜 필요한가?

다음과 같은 문제가 주어졌다고 가정해 보자.

주민등록번호를 포함하고 있는 텍스트가 있다. 이 텍스트에 포함된 모든 주민등록번호의 뒷자리를 \* 문자로 변경하시오.

우선, 정규식을 전혀 모르면 다음과 같은 순서로 프로그램을 작성해야 할 것이다.

1. 전체 텍스트를 공백 문자로 나눈다(split).
2. 나누어진 단어들이 주민등록번호 형식인지 조사한다.
3. 단어가 주민등록번호 형식이라면 뒷자리를 \*로 변환한다.
4. 나누어진 단어들을 다시 조립한다.

이를 구현한 코드는 아마도 다음과 같을 것이다.

```
data = """
park 800905-1049118
kim 700905-1059119
"""

result = []
for line in data.split("\n"):
    word_result = []
    for word in line.split(" "):
        if len(word) == 14 and word[:6].isdigit() and word[7:].isdigit():
            word = word[:6] + "-" + "*****"
            word_result.append(word)
        result.append(" ".join(word_result))
    print("\n".join(result))
```

결과값:

```
park 800905-*****
kim 700905-*****
```

반면에, 정규식을 사용하면 다음처럼 훨씬 간편하고 직관적인 코드를 작성할 수 있다. 아직 정규식 사용 방법을 배우지 않았으니 눈으로만 살펴보자.

```
import re
```



```
data = """
park 800905-1049118
kim 700905-1059119
"""

pat = re.compile("(\\d{6})[-]\\d{7}")
print(pat.sub("\\d{6}<1>-*****", data))
```

결과값:

```
park 800905-*****
kim 700905-*****
```

정규 표현식을 사용하면 이렇게 간단한 예제에서도 코드가 상당히 간결해진다. 만약 찾고자하는 문자열 또는 바꾸어야 할 문자열의 규칙이 매우 복잡하다면 정규식의 효용은 더 커지게 된다.